

实验题目：流水线 MIPS 处理器

班级：无 32

学号：2023010504

姓名：张乐

日期：2025 年 7 月 10 日

一、实验目的

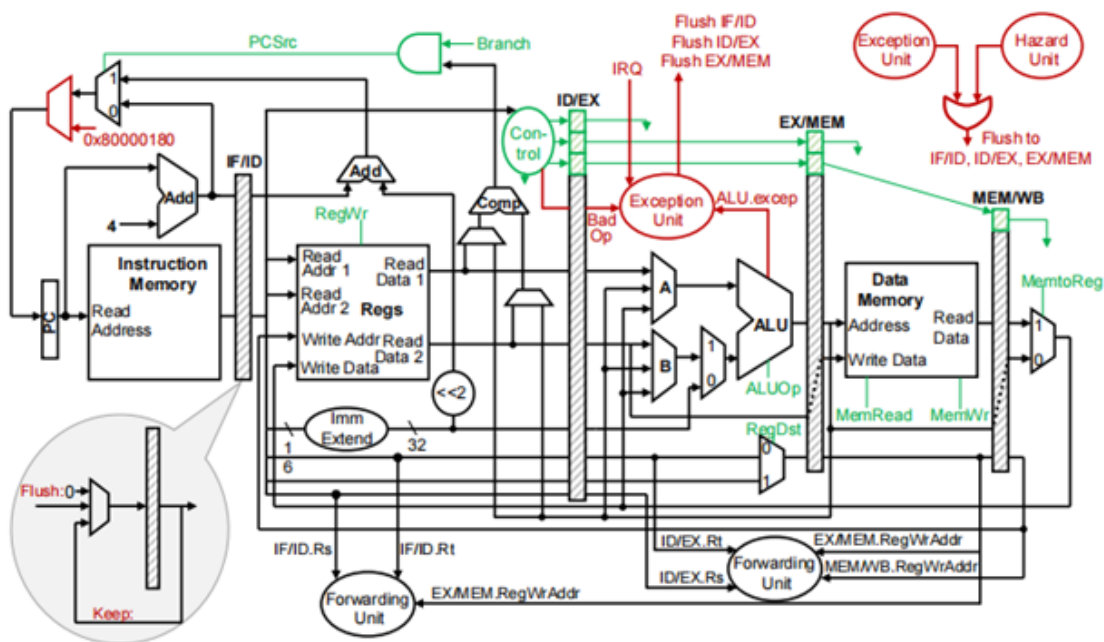
1. 将理论课处理器大作业中设计的单周期 MIPS 处理器改进为流水线结构，并利用此处理器完成稀疏矩阵乘法。
2. 学习、理解并掌握流水线处理器的工作流程，掌握外设控制、冒险和数据关联等核心知识内容。
3. 提升 Verilog 代码水平，锻炼使用 FPGA 进行工程应用的能力。

二、设计方案

1. 整体概述

MIPS 五级流水线的设计原理是通过将指令执行过程分解为五个独立的执行阶段，依靠级间寄存器实现各级之间数据与控制信号的传递，通过转发 forwarding、清空 flush、阻塞 stall 等单元消除各类冒险，利用流水线技术提升处理器性能。

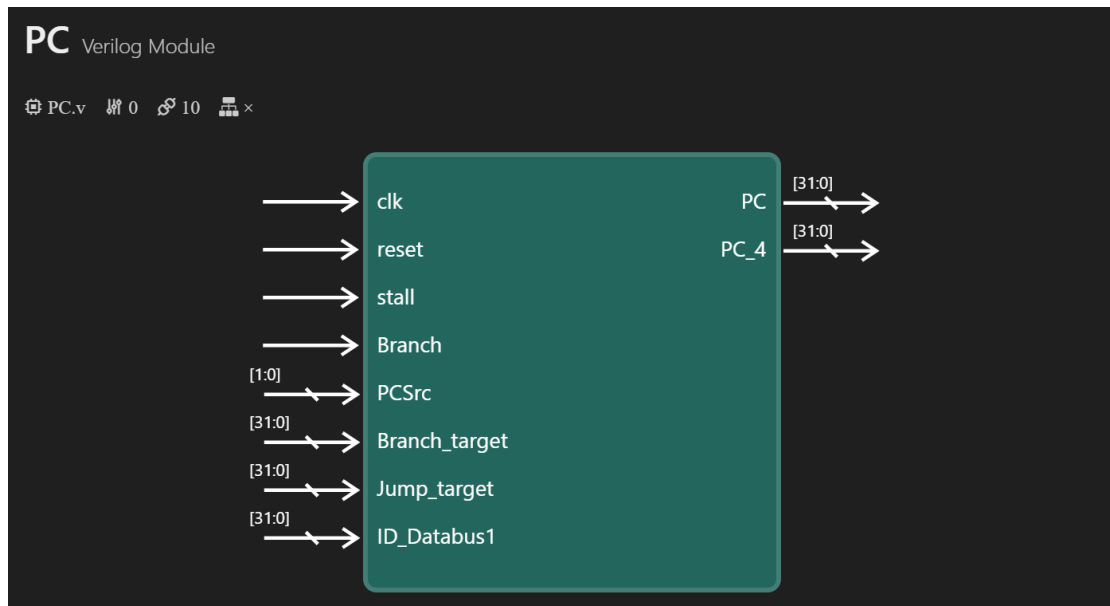
因此，MIPS 五级流水线可主要拆分为 5 个执行阶段、4 个级间寄存器、x 个冒险处理单元共 x 个模块。



2. 执行阶段

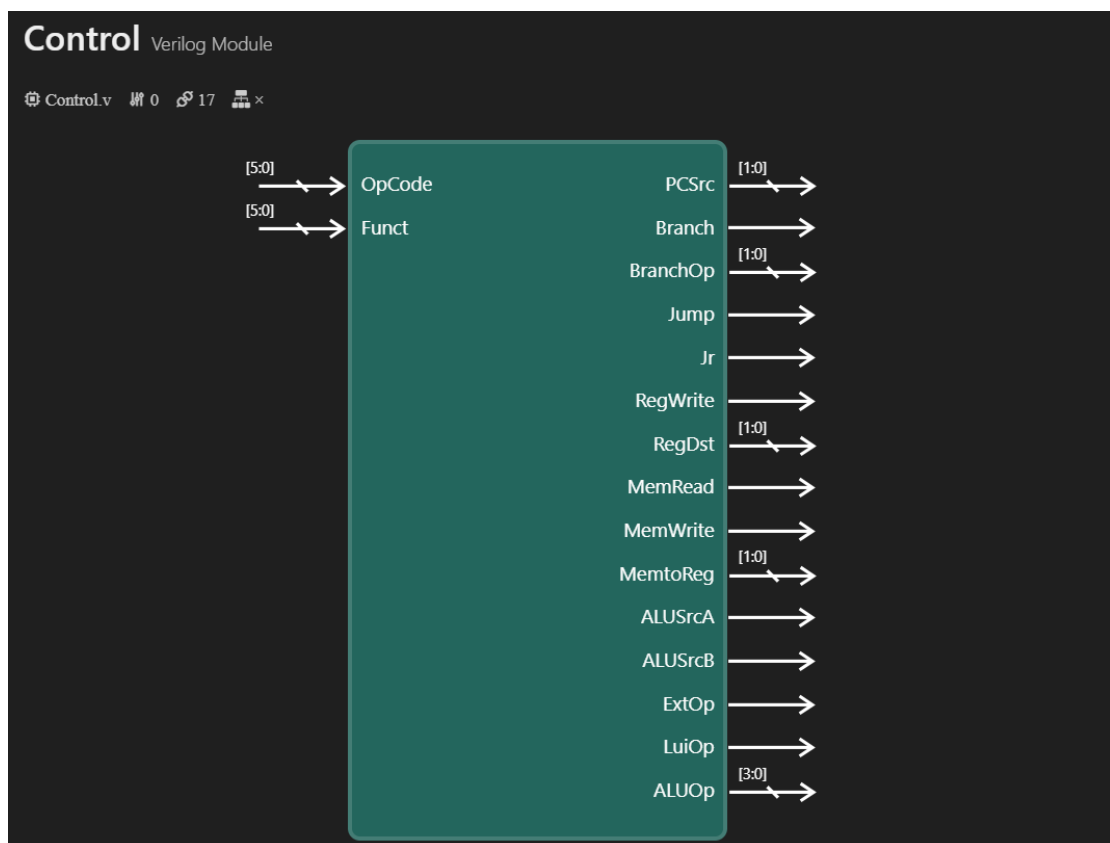
(1) IF 阶段

从指令存储器取指令，更新 PC。



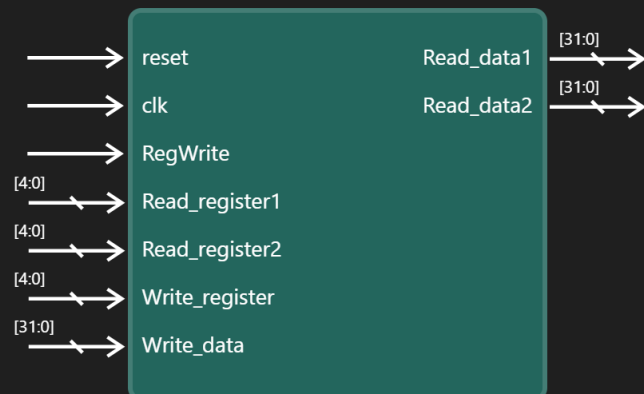
(2) ID 阶段

解码指令，进行控制信号译码，读取寄存器堆数据，并对立即数进行扩展。



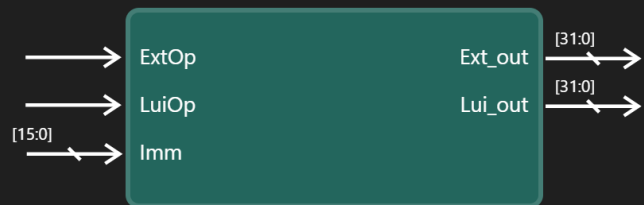
RegisterFile Verilog Module

RegisterFile.v 0 9 ×



ImmExt Verilog Module

ImmExt.v 0 5 ×



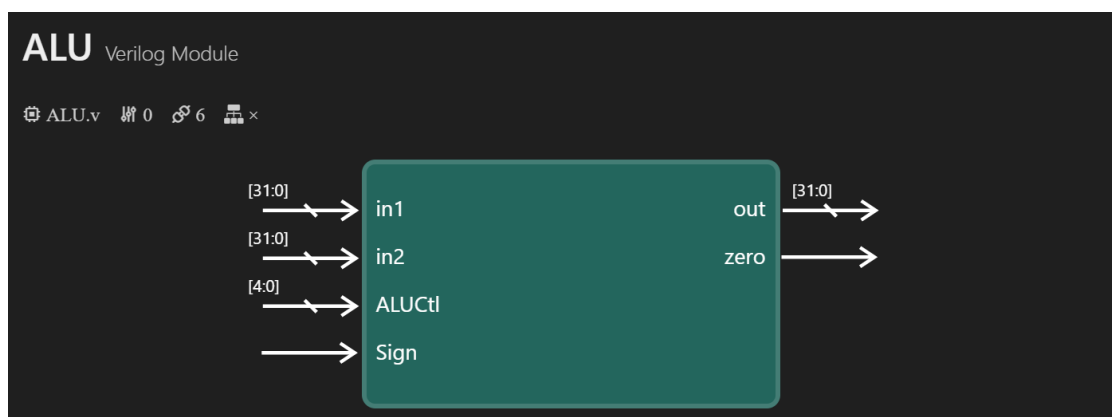
(3) EX 阶段

主要实现 ALU 操作，执行算术/逻辑运算。

ALUCtl Verilog Module

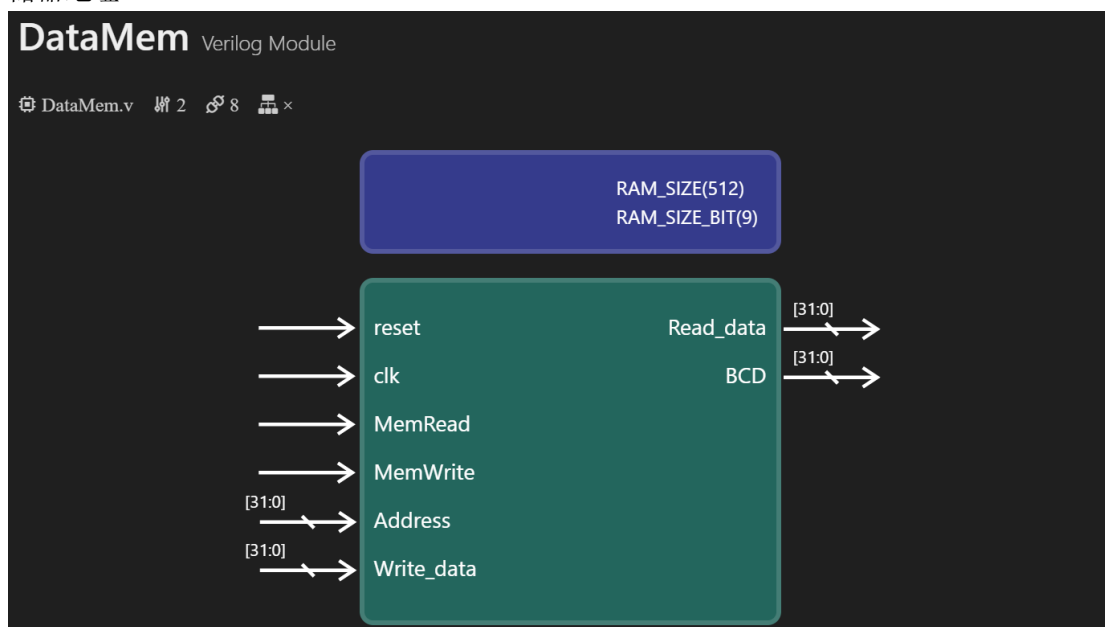
ALUCtl.v 11 4 ×





(4) MEM 阶段

访问数据存储器，并由前面阶段的结果决定是否读取或写入，以及对应该读取或写入的存储器地址。



(5) WB 阶段

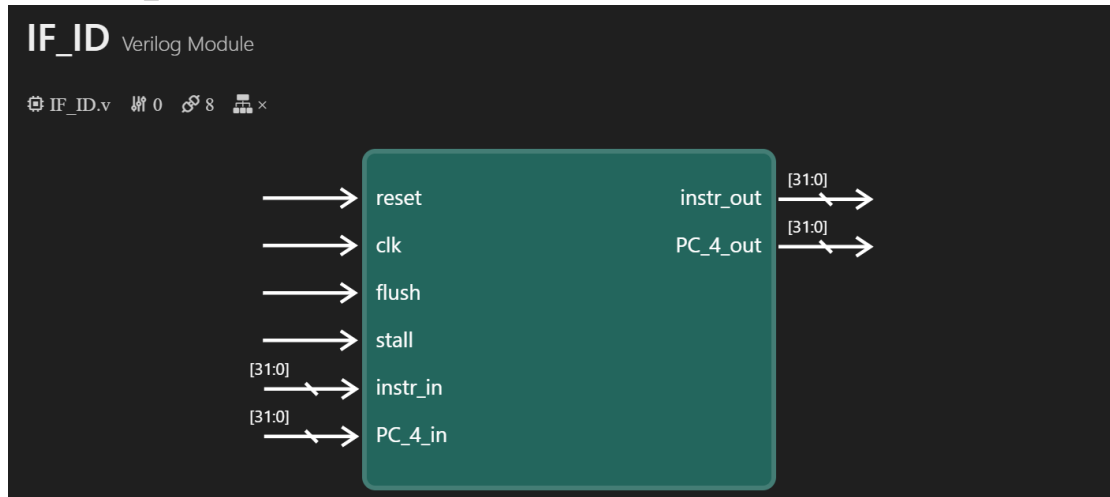
将结果写回寄存器堆，由前面阶段生成的 MemtoReg 信号控制写回的内容（计算结果、读取结果或 PC），由 RegDst 信号控制写回的位置（Rt、Rd 或其他）。

WB 阶段实现寄存器堆写入的原理框图与 ID 阶段实现寄存器堆读取的 RegisterFile 原理框图一致，故不再赘述。

3. 级间寄存器

级间寄存器级主要功能为实现流水线的向下传递，将上一级输入的数据、计算结果、控制信号中有传递必要的内容作为输出，在下一个时钟的上升沿到来时触发，传入下一级流水线。

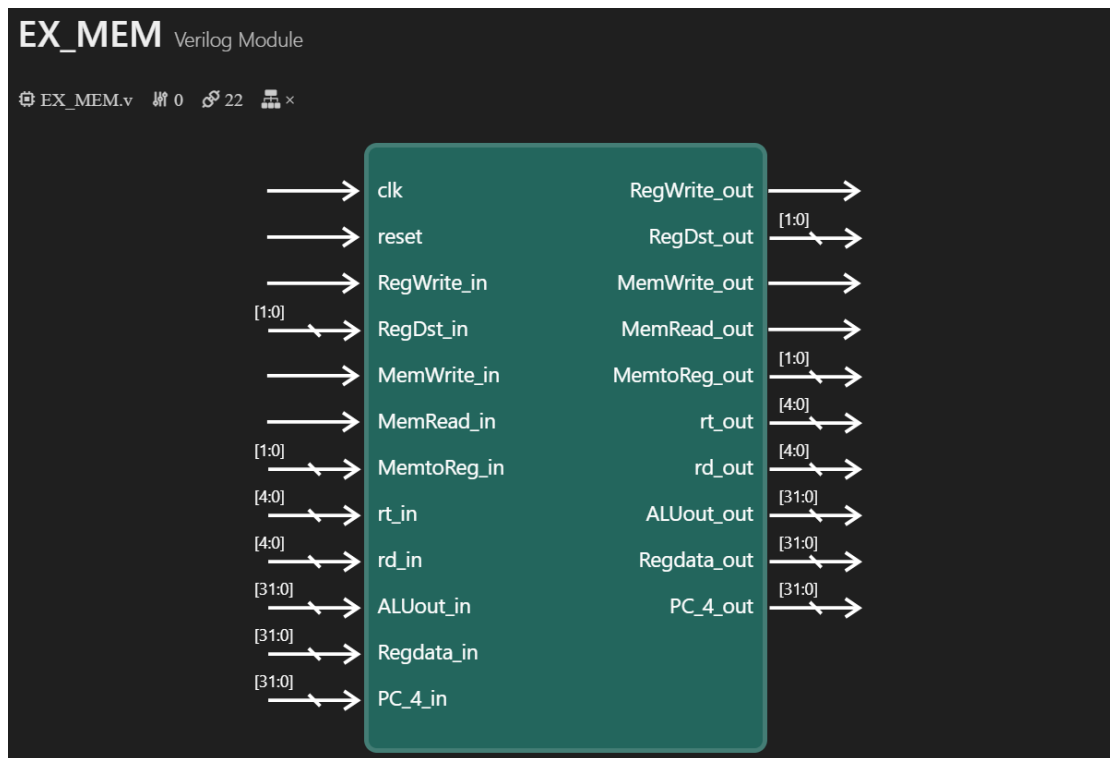
(1) IF_ID



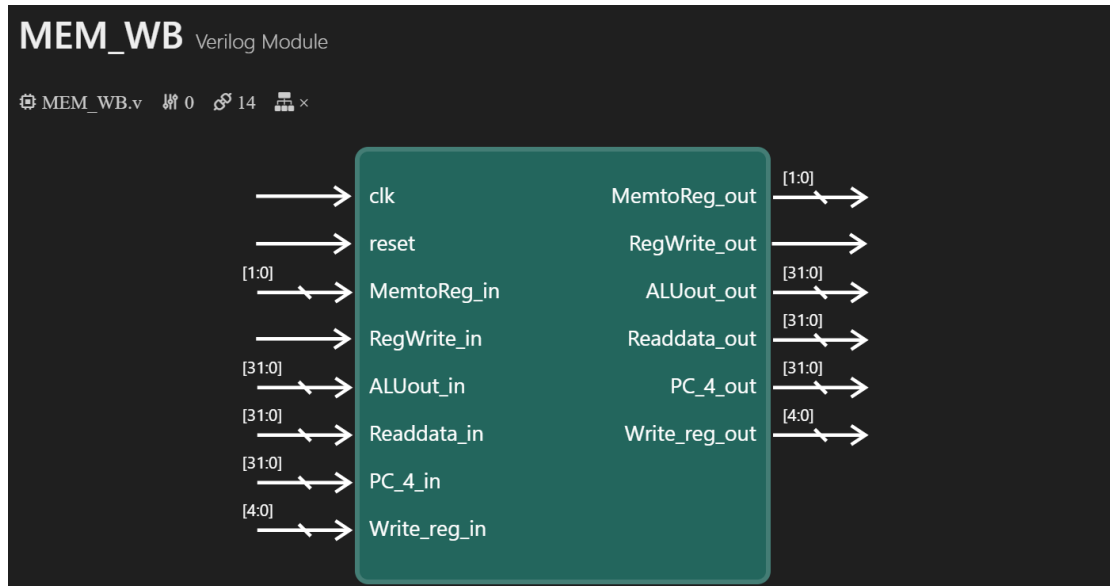
(2) ID_EX



(3) EX_MEM



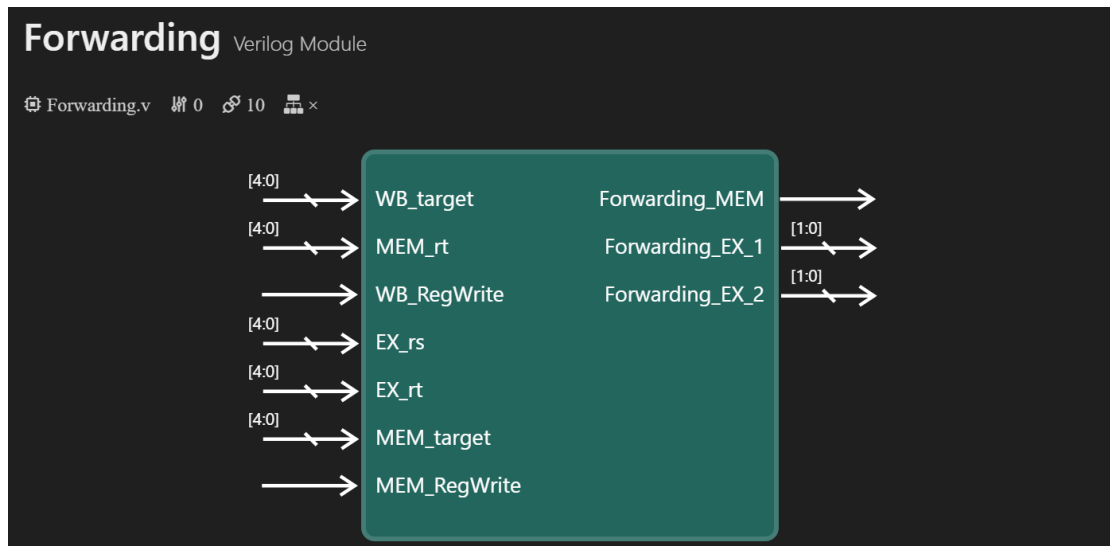
(4) MEM_WB



4. 冒险处理

(1) 转发单元 Forwarding

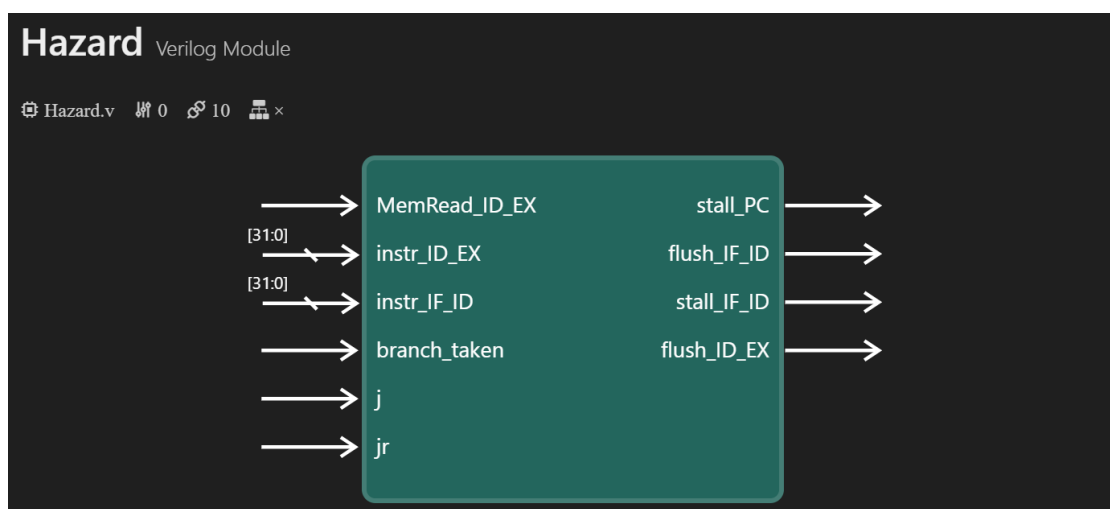
转发单元生成 3 个转发信号，其中 2 个信号用于 EX/MEM->EX 的转发，只是针对不同的寄存器 Rs 和 Rt，另一个信号用于 MEM/WB->EX 的转发。



(2) 控制冒险单元 Hazard

生成 4 个信号，针对分支指令、跳转指令、load-use 冒险。

	stall_PC	flush_IF_ID	stall_IF_ID	flush_ID_EX
正常情况	0	0	0	0
分支指令	0	1	0	0
跳转指令	0	1	0	0
load-use 冒险	1	0	1	1



三、 算法指令

1. 汇编代码

汇编代码中包括稀疏矩阵乘法和 BCD 显示两部分。完成汇编代码后，用 Mars 汇编后生成 32bit 机器码，并将指令预先存入指令存储器 InstMem 中。

(1) 稀疏矩阵乘法

稀疏矩阵乘法的核心原理是通过压缩存储格式仅处理非零元素来优化计算过程。该算法使用三个关键数组来表示稀疏矩阵：**values** 数组存储非零元素值，**col_indices** 数组记录对应的列索引，**row_ptr** 数组标记每行非零元素的起始位置。计算时首先遍历稀疏矩阵的每一行，通过 **row_ptr** 确定每行的非零元素范围，然后对每个非零元素根据其列索引找到稠密矩阵的对应行，将该非零元素与稠密矩阵对应行的所有元素相乘，并将结果累加到输出矩阵的相应位置。

(2) BCD 显示

BCD 显示的核心原理是通过动态扫描方式在数码管上轮流显示多位数字。该算法首先通过移位和掩码操作依次提取每一位的数字值，然后使用预定义的段码表将这些数字值转换为对应的 7 段数码管控制信号。接着，依次激活每个数码管的位选信号（AN0-AN3），将对应的段码和位选信号组合输出到数码管。每个数字显示约 1ms 后快速切换到下一位，利用人眼的视觉暂留效应，在高速刷新下呈现出所有数字同时显示的效果。

(3) 实际代码

```

1  .data
2      buffer:
3          # 稀疏矩阵A的元数据 [m, n, p, s]
4          .word 3 4 5 4      # m: A行数, n: A列数, p: B列数, s: A非零元素数
5          # CSR格式存储的稀疏矩阵A
6          .word 9, 7, 15, 9   # values: 非零元素值
7          .word 2, 1, 0, 2    # col_indices: 各非零元素的列索引
8          .word 0, 1, 2, 4    # row_ptr: 每行起始位置指针
9          # 稠密矩阵B (4行×5列)
10         .word 1, 4, 0, 12, 11 # B[0][0]-B[0][4]
11         .word 9, 0, 11, 8, 2  # B[1][0]-B[1][4]
12         .word 12, 2, 11, 10, 0 # B[2][0]-B[2][4]
13         .word 10, 12, 0, 1, 9  # B[3][0]-B[3][4]
14         # 数码管段码表 (0-F共16个数字)
15         seg_table: .word 0x0000003F, 0x00000006, 0x0000005B, 0x0000004F # 0-3
16                   .word 0x00000066, 0x0000006D, 0x0000007D, 0x00000007 # 4-7
17                   .word 0x0000007F, 0x0000006F, 0x00000077, 0x0000007C # 8-B
18                   .word 0x00000039, 0x0000005E, 0x00000079, 0x00000071 # C-F
19         # 结果矩阵C (3行×5列, 预分配空间)
20         C: .space 60        # 3*5*4=60字节

```

```

22 .text
23 main:
24     jal sparse_matmul      # 调用稀疏矩阵乘法函数
25
26     # 显示结果矩阵C
27     la $a0, C              # $a0 = 矩阵C基地址
28     la $t0, buffer         # 加载buffer地址
29     lw $a1, 0($t0)         # $a1 = m (行数)
30     lw $a2, 8($t0)         # $a2 = p (列数)
31     jal display_matrix_bcd # 调用数码管显示函数
32
33     # =====
34     # 数码管显示函数 - 按行优先显示矩阵元素 (每个数字显示1秒)
35     # 参数: $a0=矩阵地址, $a1=行数, $a2=列数
36     # =====
37 display_matrix_bcd:
38     move $t0, $a0          # $t0 = 矩阵基地址
39     li $t1, 0              # $t1 = 行计数器(i)
40     lui $t7, 0x4000        # 数码管地址高16位 (MMIO)
41     ori $t7, $t7, 0x0010   # 数码管地址低16位 (0x40000010)
42
43 display_row_loop:
44     beq $t1, $a1, display_done # 所有行处理完成则结束
45     li $t2, 0              # $t2 = 列计数器(j)
46
47 display_col_loop:
48     beq $t2, $a2, next_row  # 当前行所有列处理完成
49
50     # 计算当前元素地址 C[i][j]
51     mul $t3, $t1, $a2       # i * 列数
52     add $t3, $t3, $t2       # + j
53     sll $t3, $t3, 2         # 字偏移量 (×4)
54     add $t3, $t0, $t3       # 元素地址 = 基址 + 偏移
55     lw $a0, 0($t3)         # $a0 = 当前元素值
56
57     # 数码管动态扫描 (250次×4ms=1秒)
58     li $s0, 250            # 循环计数器
59
60 number_display_loop:
61     li $t4, 0              # $t4 = 位计数器(0-3)
62     la $t9, seg_table      # $t9 = 段码表基址
63
64 digit_refresh:
65     beq $t4, 4, refresh_done # 4位扫描完成
66

```

```

67      # 提取当前4位数字 (替代srlv指令)
68      beq $t4, 0, get_digit0 # 取最低4位
69      beq $t4, 1, get_digit1 # 取次低4位
70      beq $t4, 2, get_digit2 # 取次高4位
71      beq $t4, 3, get_digit3 # 取最高4位
72
73  get_digit0:
74      andi $t8, $a0, 0xF      # 取bit0-3
75      j digit_processed
76
77  get_digit1:
78      srl $t8, $a0, 4         # 右移4位后取bit0-3
79      andi $t8, $t8, 0xF
80      j digit_processed
81
82  get_digit2:
83      srl $t8, $a0, 8         # 右移8位后取bit0-3
84      andi $t8, $t8, 0xF
85      j digit_processed
86
87  get_digit3:
88      srl $t8, $a0, 12        # 右移12位后取bit0-3
89      andi $t8, $t8, 0xF
90
91  digit_processed:
92      # 查询段码表
93      sll $t5, $t8, 2         # 索引×4 (段码表为word数组)
94      add $t5, $t9, $t5       # 段码地址 = 基址 + 偏移
95      lw $t5, 0($t5)          # 加载段码值
96      andi $t5, $t5, 0xFF     # 取有效8位段码
97
98      # 生成位选信号 (替代sllv指令)
99      beq $t4, 0, set_an0     # AN0选择最低位数字
100     beq $t4, 1, set_an1     # AN1
101     beq $t4, 2, set_an2     # AN2
102     beq $t4, 3, set_an3     # AN3选择最高位数字
103
104  set_an0:
105      li $t6, 0x0100          # AN0掩码
106      j combine
107  set_an1:
108      li $t6, 0x0200          # AN1掩码
109      j combine
110  set_an2:
111      li $t6, 0x0400          # AN2掩码
112      j combine
113  set_an3:
114      li $t6, 0x0800          # AN3掩码

```

```

116 combine:
117     or $t5, $t5, $t6      # 组合段码和位选信号
118
119     # 输出到数码管
120     sw $t5, 0($t7)        # 写入数码管寄存器
121
122     # 精确1ms延时 (假设CPU频率100MHz)
123     li $s1, 6250          # 循环次数 (经测试校准)
124 delay_loop:
125     addi $s1, $s1, -1
126     bne $s1, $zero, delay_loop
127
128     addi $t4, $t4, 1      # 处理下一位
129     j digit_refresh
130
131 refresh_done:
132     addi $s0, $s0, -1     # 完成1次4位扫描(4ms)
133     bne $s0, $zero, number_display_loop # 未达250次则继续
134
135     # 当前数字显示满1秒后切换
136     addi $t2, $t2, 1      # 下一列
137     j display_col_loop
138
139 next_row:
140     addi $t1, $t1, 1      # 下一行
141     j display_row_loop
142
143 display_done:
144     jr $ra                # 返回调用处
145
146 # =====
147 # 稀疏矩阵乘法函数 ( $A[m \times n] * B[n \times p] = C[m \times p]$ )
148 # 使用CSR格式存储稀疏矩阵A
149 # =====
150 sparse_matmul:
151     # 加载矩阵参数
152     la $t0, buffer        # buffer基地址
153     lw $s0, 0($t0)         # $s0 = m (A行数)
154     lw $s1, 4($t0)         # $s1 = n (A列数/B行数)
155     lw $s2, 8($t0)         # $s2 = p (B列数)
156     lw $s3, 12($t0)        # $s3 = s (A非零元素数)
157
158     # 初始化CSR结构指针
159     addi $s4, $t0, 16      # $s4 = values数组地址
160     sll $s5, $s3, 2        # 计算col_indices偏移
161     add $s5, $s5, $s4      # $s5 = col_indices数组地址
162     sll $s6, $s3, 3        # 计算row_ptr偏移 (s3×8)
163     add $s6, $s6, $s4      # $s6 = row_ptr数组地址

```

```

165 # 计算矩阵B基地址
166 addi $s7, $s0, 1 # row_ptr有m+1个元素
167 sll $s7, $s7, 2 # 计算B偏移 (m+1)×4
168 add $s7, $s7, $s6 # $s7 = B矩阵基地址
169
170 # 初始化结果矩阵C为零
171 li $t0, 0 # $t0 = 元素计数器
172 la $a1, C # $a1 = C矩阵基地址
173 mul $t1, $s0, $s2 # $t1 = 总元素数(m×p)
174
175 init_loop:
176 beq $t0, $t1, end_init # 初始化完成
177 sll $t2, $t0, 2 # 计算偏移
178 add $t3, $t2, $a1 # 当前元素地址
179 sw $zero, 0($t3) # C[i] = 0
180 addi $t0, $t0, 1 # 计数器+1
181 j init_loop
182
183 end_init:
184 # 开始稀疏矩阵乘法
185 li $t0, 0 # $t0 = i (A行号)
186
187 outer_loop:
188 beq $t0, $s0, end_outer # 所有行处理完成
189
190 # 获取当前行非零元素范围
191 sll $t2, $t0, 2 # i×4
192 add $t2, $t2, $s6 # row_ptr[i]地址
193 lw $t3, 0($t2) # $t3 = start (行起始索引)
194 lw $t4, 4($t2) # $t4 = end (行结束索引)
195 move $t5, $t3 # $t5 = j (非零元素计数器)
196
197 inner_loop:
198 beq $t5, $t4, end_inner # 当前行所有非零元素处理完成
199
200 # 加载A的非零元素信息
201 sll $t2, $t5, 2 # j×4
202 add $t8, $t2, $s5 # col_indices[j]地址
203 lw $t6, 0($t8) # $t6 = k (A的列号/B的行号)
204 add $t8, $t2, $s4 # values[j]地址
205 lw $t7, 0($t8) # $t7 = A[i][k]的值
206
207 # 计算C[i][*] += A[i][k] * B[k][*]
208 li $t9, 0 # $t9 = l (B列号)
209
210 b_col_loop:
211 beq $t9, $s2, end_b_col # 所有B列处理完成
212

```

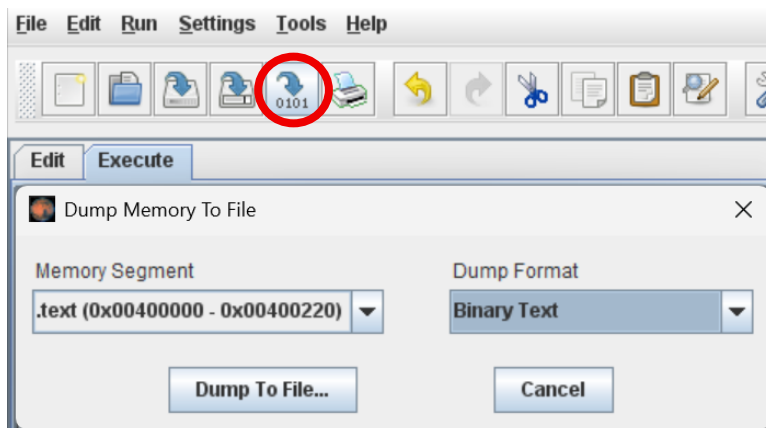


```

213      # 计算C[i][1]地址
214      mul $t8, $t0, $s2      # i × p
215      add $t8, $t8, $t9      # + 1
216      sll $t8, $t8, 2        # 字偏移
217      add $t8, $t8, $a1      # C[i][1]地址
218      lw $t2, 0($t8)         # 加载当前C[i][1]值
219
220      # 计算B[k][1]地址
221      mul $t1, $t6, $s2      # k × p
222      add $t1, $t1, $t9      # + 1
223      sll $t1, $t1, 2        # 字偏移
224      add $t1, $t1, $s7      # B[k][1]地址
225      lw $t1, 0($t1)         # 加载B[k][1]值
226
227      # 累加结果
228      mul $t1, $t1, $t7      # A[i][k] * B[k][1]
229      add $t2, $t2, $t1      # C[i][1] += 乘积
230      sw $t2, 0($t8)         # 存回C[i][1]
231
232      addi $t9, $t9, 1        # 下一列
233      j b_col_loop
234
235  end_b_col:
236      addi $t5, $t5, 1        # 处理A的下一个非零元素
237      j inner_loop
238
239  end_inner:
240      addi $t0, $t0, 1        # 下一行
241      j outer_loop
242
243  end_outer:
244      mul $v0, $s0, $s2      # 返回值 = 总元素数(m×p)
245      jr $ra                  # 返回调用处

```

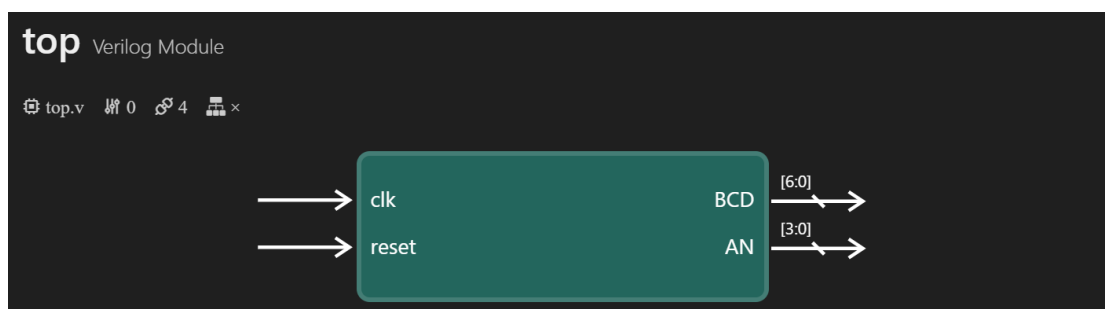
汇编代码中可加入输出语句调试，确认计算出的结果矩阵和写入的 BCD 码均正确后，可利用汇编生成二进制 txt 形式的机器码。



2. 硬件逻辑设计

关于流水线中各环节的硬件逻辑已在前文说明，此处主要说明针对稀疏矩阵乘法和 BCD 显示算法的硬件设计。为实现稀疏矩阵乘法，需要在数据存储器 DataMem 中事先写入矩阵信息和 BCD 段码表，并在指令存储器 InstMem 中写入由只保留了计算和显示核心部分（无文件读取、文件写入、程序结束）的汇编代码生成的机器码。此外，还需要用 Verilog 代码将 CPU 和 BCD 显示的硬件逻辑共同封装在顶层文件 top 中。

除此之外，由于时钟频率需求，top 文件中还包括二分频功能，以支持流水线在硬件开发板上的正常运行。



四、 关键代码及文件清单

1. 关键代码

(1) IF 阶段：计算 PC 和取指令

```
wire [31:0] PC_next =  
    (Branch) ? Branch_target:  
    (PCSrc == 2'b01) ? Jump_target:  
    (PCSrc == 2'b10) ? ID_Databus1:  
    PC_4;  
  
always @(posedge clk or posedge reset) begin  
    if (reset) PC <= 32'h00000000;  
    else if (~stall) PC <= PC_next;  
end  
  
assign PC_4 = PC + 32'd4;
```

```
wire [31:0] IF_instr;  
  
InstMem myInstMem(  
    .Address    (IF_PC    ),
```

```
.instr      (IF_instr    )  
);
```

(2) ID 阶段：生成控制信号、寄存器堆读取、立即数扩展

```
assign PCSrc =  
    (OpCode == 6'h02 || OpCode == 6'h03)? 2'b01: //j or jal  
    (OpCode == 6'h00 && Funct == 6'h08)? 2'b10: //jr  
    2'b00; //others  
  
assign Branch = (OpCode == 6'b000100 || // beq  
    OpCode == 6'b000101 || // bne  
    OpCode == 6'b000110 || // blez  
    OpCode == 6'b000111 || // bgtz  
    OpCode == 6'h0d); // bltz  
  
// 分支类型编码  
assign BranchOp = (OpCode == 6'b000100) ? 2'b00 : // beq  
    (OpCode == 6'b000101) ? 2'b01 : // bne  
    2'b10; // blez/bgtz/bltz  
  
assign Jump = (OpCode == 6'b000010 || // j  
    OpCode == 6'b000011); // jal  
  
assign Jr = (OpCode == 6'b000000 && // R-type  
    (Funct == 6'b001000 || // jr  
    Funct == 6'b001001)); // jalr  
  
assign RegWrite =  
    (OpCode == 6'h2b || OpCode == 6'h04 || OpCode == 6'h02 ||  
(OpCode == 6'h00 && Funct == 6'h08))? 1'b0: //sw, beq, j, jr  
    1'b1; //others  
  
assign RegDst =  
    (OpCode == 6'h00 || OpCode == 6'h1c)? 2'b01: //R, mul  
    (OpCode == 6'h03)? 2'b10: //jal  
    2'b00; //others  
  
assign MemRead =  
    (OpCode == 6'h23)? 1'b1: //lw  
    1'b0; //others  
  
assign MemWrite =  
    (OpCode == 6'h2b)? 1'b1: //sw  
    1'b0; //others
```



```
assign MemtoReg =
    (OpCode == 6'h23)? 2'b01: //lw
    (OpCode == 6'h03)? 2'b10: //jal
    2'b00; //others

assign ALUSrcA =
    (OpCode == 6'h00 && (Funct == 6'h00 || Funct == 6'h02 || Funct
== 6'h03))? 1'b1: //sll, slrl, sra
    1'b0; //others

assign ALUSrcB =
    (OpCode == 6'h00 || OpCode == 6'h1c || OpCode == 6'h04)? 1'b0:
//R, mul, eq
    1'b1; //others

assign ExtOp =
    (OpCode == 6'h0c)? 1'b0: //andi
    1'b1; //others

assign LuiOp =
    (OpCode == 6'h0f)? 1'b1: //lui
    1'b0; //others

// set ALUOp
assign ALUOp[2:0] =
    (OpCode == 6'h00)? 3'b010:
    (OpCode == 6'h04)? 3'b001:
    (OpCode == 6'h0c)? 3'b100:
    (OpCode == 6'h0a || OpCode == 6'h0b)? 3'b101:
    (OpCode == 6'h1c && Funct == 6'h02)? 3'b110:
    3'b000; //mul

assign ALUOp[3] = OpCode[0];
```

```
// read data from RF_data as Read_data1 and Read_data2
assign Read_data1 = (Read_register1 == 5'b00000)? 32'h00000000:
    (Read_register1 == Write_register)? Write_data:
    RF_data[Read_register1];
assign Read_data2 = (Read_register2 == 5'b00000)? 32'h00000000:
    (Read_register2 == Write_register)? Write_data:
    RF_data[Read_register2];

integer i;
// write Wrtie_data to RF_data at clock posedge
```

```
always @(posedge reset or posedge clk)
    if (reset)
        for (i = 1; i < 32; i = i + 1)
            RF_data[i] <= 32'h00000000;
    else if (RegWrite && (Write_register != 5'b00000))
        RF_data[Write_register] <= Write_data;
```

```
assign Ext_out = {(ExtOp)? {16{Imm[15]}}: 16'h0000, Imm};
assign Lui_out = LuiOp? {Imm, 16'h0000}: Ext_out;
```

(3) EX 阶段: ALU

```
//后续准备工作
assign EX_rs = EX_instr[25:21];
assign EX_rt = EX_instr[20:16];
assign EX_rd = EX_instr[15:11];
assign EX_shamt = EX_instr[10:6];
assign EX_Regdata1 = (EX_Forwarding_alu1[0]) ? MEM_ALUout:
                    (EX_Forwarding_alu1[1]) ? WB_Writedata:
EX_Databus1;
assign EX_Regdata2 = (EX_Forwarding_alu2[0]) ? MEM_ALUout:
                    (EX_Forwarding_alu2[1]) ? WB_Writedata:
EX_Databus2;
assign ALU_in1 = (EX_ALUSrcA)? {27'h00000, EX_shamt}: EX_Regdata1;
assign ALU_in2 = (EX_ALUSrcB)? EX_Imm: EX_Regdata2;
```

```
//zero means whether the output is zero or not
assign zero = (in1 == in2);
```

```
wire ss;
assign ss = {in1[31], in2[31]};
```

```
wire lt_31;
assign lt_31 = (in1[30:0] < in2[30:0]);
```

```
//lt_signed means whether (in1 < in2)
wire lt_signed;
assign lt_signed = (in1[31] ^ in2[31])?
    ((ss == 2'b01)? 0: 1): lt_31;
```

```
//different ALU operations
```

```
always @(*)
    case (ALUCtl)
        5'b00000: out <= in1 & in2;
```

```
5'b00001: out <= in1 | in2;
5'b00010: out <= in1 + in2;
5'b00110: out <= in1 - in2;
5'b00111: out <= {31'h00000000, Sign? lt_signed: (in1 <
in2)};

5'b01100: out <= ~(in1 | in2);
5'b01101: out <= in1 ^ in2;
5'b10000: out <= (in2 << in1[4:0]);
5'b11000: out <= (in2 >> in1[4:0]);
5'b11001: out <= ({32{in2[31]}}, in2) >> in1[4:0]);
5'b11010: out <= in1 * in2; //mul
default: out <= 32'h00000000;
endcase
```

```
assign IF_branch = (EX_branch == 1'b0) ? 1'b0 : // 不分支
                   (EX_branchOp == 2'b00) ? zero : // beq: rs1 == rs2
                   (EX_branchOp == 2'b01) ? ~zero : // bne: rs1 != rs2
                   1'b0; // 其他情况默认不分支
```

(4) MEM 阶段: DataMem 读取与写入

```
always @(posedge reset or posedge clk) begin
    if (reset) begin
        // 1. 存储参数
        RAM_data[0] <= 32'h00000003; // M = 3
        RAM_data[1] <= 32'h00000004; // N = 4
        RAM_data[2] <= 32'h00000005; // P = 5
        RAM_data[3] <= 32'h00000004; // S = 4

        // 2. values 数组
        RAM_data[4] <= 32'h00000009; // 9
        RAM_data[5] <= 32'h00000007; // 7
        RAM_data[6] <= 32'h0000000f; // 15
        RAM_data[7] <= 32'h00000009; // 9

        // 3. col_indices 数组
        RAM_data[8] <= 32'h00000002; // 2
        RAM_data[9] <= 32'h00000001; // 1
        RAM_data[10] <= 32'h00000000; // 0
        RAM_data[11] <= 32'h00000002; // 2

        // 4. row_ptr 数组
        RAM_data[12] <= 32'h00000000; // 0
        RAM_data[13] <= 32'h00000001; // 1
        RAM_data[14] <= 32'h00000002; // 2
```

```
RAM_data[15] <= 32'h00000004; // 4

// 5. 稠密矩阵 B
// 第 0 行
RAM_data[16] <= 32'h00000001; // B[0][0] = 1
RAM_data[17] <= 32'h00000004; // B[0][1] = 4
RAM_data[18] <= 32'h00000000; // B[0][2] = 0
RAM_data[19] <= 32'h0000000C; // B[0][3] = 12
RAM_data[20] <= 32'h0000000B; // B[0][4] = 11

// 第 1 行
RAM_data[21] <= 32'h00000009; // B[1][0] = 9
RAM_data[22] <= 32'h00000000; // B[1][1] = 0
RAM_data[23] <= 32'h0000000B; // B[1][2] = 11
RAM_data[24] <= 32'h00000008; // B[1][3] = 8
RAM_data[25] <= 32'h00000002; // B[1][4] = 2

// 第 2 行
RAM_data[26] <= 32'h0000000C; // B[2][0] = 12
RAM_data[27] <= 32'h00000002; // B[2][1] = 2
RAM_data[28] <= 32'h0000000B; // B[2][2] = 11
RAM_data[29] <= 32'h0000000A; // B[2][3] = 10
RAM_data[30] <= 32'h00000000; // B[2][4] = 0

// 第 3 行
RAM_data[31] <= 32'h0000000A; // B[3][0] = 10
RAM_data[32] <= 32'h0000000C; // B[3][1] = 12
RAM_data[33] <= 32'h00000000; // B[3][2] = 0
RAM_data[34] <= 32'h00000001; // B[3][3] = 1
RAM_data[35] <= 32'h00000009; // B[3][4] = 9

// 段码表
RAM_data[36] <= 32'h0000003F;
RAM_data[37] <= 32'h00000006;
RAM_data[38] <= 32'h0000005B;
RAM_data[39] <= 32'h0000004F;
RAM_data[40] <= 32'h00000066;
RAM_data[41] <= 32'h0000006D;
RAM_data[42] <= 32'h0000007D;
RAM_data[43] <= 32'h00000007;
RAM_data[44] <= 32'h0000007F;
RAM_data[45] <= 32'h0000006F;
RAM_data[46] <= 32'h00000077;
RAM_data[47] <= 32'h0000007C;
```

```
RAM_data[48] <= 32'h00000039;
RAM_data[49] <= 32'h0000005E;
RAM_data[50] <= 32'h00000079;
RAM_data[51] <= 32'h00000071;

// 清零内存
for (i = 52; i < RAM_SIZE; i = i + 1)
    RAM_data[i] <= 32'h00000000;

end else if (MemWrite) begin
    case (Address)
        32'h40000010: BCD <= Write_data;
        default: RAM_data[Address[RAM_SIZE_BIT + 1:2]] <=
Write_data;
    endcase
end
end
```

(5) WB 阶段：寄存器堆写入

```
// read data from RF_data as Read_data1 and Read_data2
assign Read_data1 = (Read_register1 == 5'b00000)? 32'h00000000:
    (Read_register1 == Write_register)? Write_data:
    RF_data[Read_register1];
assign Read_data2 = (Read_register2 == 5'b00000)? 32'h00000000:
    (Read_register2 == Write_register)? Write_data:
    RF_data[Read_register2];

integer i;
// write Write_data to RF_data at clock posedge
always @(posedge reset or posedge clk)
    if (reset)
        for (i = 1; i < 32; i = i + 1)
            RF_data[i] <= 32'h00000000;
    else if (RegWrite && (Write_register != 5'b00000))
        RF_data[Write_register] <= Write_data;
```

(6) IF_ID 级间寄存器

```
always @(posedge reset or posedge clk) begin //整体代码优先级: 重置 reset>
清空 flush>阻塞 stall>正常执行
    //重置 reset
    if(reset) begin
        instr_out <= 32'd0;
```

```
        PC_4_out <= 32'd0;
    end
    //清空 flush
    else if(flush) begin
        instr_out <= 32'd0;
        PC_4_out <= 32'd0;
    end
    //阻塞 stall
    else if(stall) begin
        instr_out <= instr_out;
        PC_4_out <= PC_4_out;
    end
    //正常执行
    else begin
        instr_out <= instr_in;
        PC_4_out <= PC_4_in;
    end
end
end
```

(7) ID_EX 级间寄存器

```
always @(posedge reset or posedge clk) begin //整体代码优先级: 重置
reset>清空 flush>阻塞 stall>正常执行
    //重置 reset
    if(reset) begin
        PC_4_out <= 32'd0;
        instr_out <= 32'd0;
        Imm_out <= 32'd0;
        Branch_out <= 1'd0;
        BranchOp_out <= 2'b11;
        RegWrite_out <= 1'd0;
        RegDst_out <= 2'd0;
        MemRead_out <= 1'd0;
        MemWrite_out <= 1'd0;
        MemtoReg_out <= 2'd0;
        ALUSrcA_out <= 1'd0;
        ALUSrcB_out <= 1'd0;
        ALUOp_out <= 4'd0;
        DatabusA_out <= 32'd0;
        DatabusB_out <= 32'd0;
    end
    //清空 flush
    else if(flush) begin
        PC_4_out <= 32'd0;
```

```
instr_out <= 32'd0;
Imm_out <= 32'd0;
Branch_out <= 1'd0;
BranchOp_out <= 2'b11;
RegWrite_out <= 1'd0;
RegDst_out <= 2'd0;
MemRead_out <= 1'd0;
MemWrite_out <= 1'd0;
MemtoReg_out <= 2'd0;
ALUSrcA_out <= 1'd0;
ALUSrcB_out <= 1'd0;
ALUOp_out <= 4'd0;
DatabusA_out <= 32'd0;
DatabusB_out <= 32'd0;
end
//正常执行
else begin
    PC_4_out <= PC_4_in;
    instr_out <= instr_in;
    Imm_out <= Imm_in;
    Branch_out <= Branch_in;
    BranchOp_out <= BranchOp_in;
    RegWrite_out <= RegWrite_in;
    RegDst_out <= RegDst_in;
    MemRead_out <= MemRead_in;
    MemWrite_out <= MemWrite_in;
    MemtoReg_out <= MemtoReg_in;
    ALUSrcA_out <= ALUSrcA_in;
    ALUSrcB_out <= ALUSrcB_in;
    ALUOp_out <= ALUOp_in;
    DatabusA_out <= DatabusA_in;
    DatabusB_out <= DatabusB_in;
end
end
```

(8) EX_MEM 级间寄存器

```
always @(posedge clk or posedge reset) begin //整体代码优先级: 重置 reset>
    正常执行
    if(reset) begin //重置 reset
        //Control
        RegWrite_out <= 1'b0;
        RegDst_out <= 2'b00;
        MemWrite_out <= 1'b0;
```

```
MemRead_out <= 1'b0;
MemtoReg_out <= 2'b00;
//Data
rt_out <= 5'd0;
rd_out <= 5'd0;
ALUout_out <= 32'd0;
Regdata_out <= 32'd0;
PC_4_out <= 32'd0;
end
//正常执行
else begin
    //Control
    RegWrite_out <= RegWrite_in;
    RegDst_out <= RegDst_in;
    MemWrite_out <= MemWrite_in;
    MemRead_out <= MemRead_in;
    MemtoReg_out <= MemtoReg_in;
    //Data
    rt_out <= rt_in;
    rd_out <= rd_in;
    ALUout_out <= ALUout_in;
    Regdata_out <= Regdata_in;
    PC_4_out <= PC_4_in;
end
end
```

(9) MEM_WB 级间寄存器

```
always @ (posedge clk or posedge reset) begin
    if(reset) begin
        RegWrite_out <= 1'b0;
        MemtoReg_out <= 2'b00; //11?
        ALUout_out <= 32'd0;
        Readdata_out <= 32'd0;
        PC_4_out <= 32'd0;
        Write_reg_out <= 5'd0;
    end
    else begin
        RegWrite_out <= RegWrite_in;
        MemtoReg_out <= MemtoReg_in;
        ALUout_out <= ALUout_in;
        Readdata_out <= Readdata_in;
        PC_4_out <= PC_4_in;
        Write_reg_out <= Write_reg_in;
    end
end
```



```
end  
end
```

(10) 转发单元

```
assign Forwarding_MEM = (WB_target == MEM_rt) && WB_RegWrite;  
assign Forwarding_EX_1 = (EX_rs == 5'b00000) ? 2'b00 :  
    (MEM_RegWrite && EX_rs == MEM_target) ? 2'b01 :  
    (WB_RegWrite && EX_rs == WB_target) ? 2'b10 :  
    2'b00;  
assign Forwarding_EX_2 = (EX_rt == 5'b00000) ? 2'b00 :  
    (MEM_RegWrite && EX_rt == MEM_target) ? 2'b01 :  
    (WB_RegWrite && EX_rt == WB_target) ? 2'b10 :  
    2'b00;
```

(11) 冒险控制单元

```
// Load-use 冒险检测  
wire load_use_hazard = MemRead_ID_EX && (Rt_ID_EX != 0) &&  
((Rt_ID_EX == Rs_IF_ID) || (Rt_ID_EX == Rt_IF_ID));  
  
// 控制冒险检测  
wire control_hazard = j || jr || branch_taken;  
  
// PC 控制 (1=停止, 0=正常更新)  
assign stall_PC = load_use_hazard;  
  
// IF/ID 寄存器控制  
assign stall_IF_ID = load_use_hazard; // 1=冻结  
assign flush_IF_ID = control_hazard; // 1=清空  
  
// ID/EX 寄存器控制  
assign flush_ID_EX = load_use_hazard || (branch_taken && !j  
&& !jr); // 1=插入气泡
```

(12) 二分频单元

```
parameter CNT = 2;  
  
reg count;  
  
always @(posedge clk or posedge reset)  
begin  
    if(reset) begin  
        clk_50M <= 1'b0;
```

```
        count <= 1'd0;
    end
    else begin
        count <= (count==CNT-1'd1) ? 16'd0 : count + 16'd1;
        clk_50M <= (count==1'd0) ? ~clk_50M : clk_50M;
    end
end
end
```

(13) 顶层实例化：分频、流水线 CPU、BCD 显示

```
clk_gen myclk_gen(
    .clk      (clk      ),
    .reset    (reset    ),
    .clk_50M  (clk_50M  )
);

CPU myCPU(
    .reset    (reset    ),
    .clk      (clk_50M  ),
    .bcd      (an_bcd   )
);

always @(posedge clk or posedge reset)
begin
    if(reset)
        BCD <= 7'd0;
    else
        BCD <= an_bcd[6:0];
        AN <= an_bcd[11:8];
    end
end
```

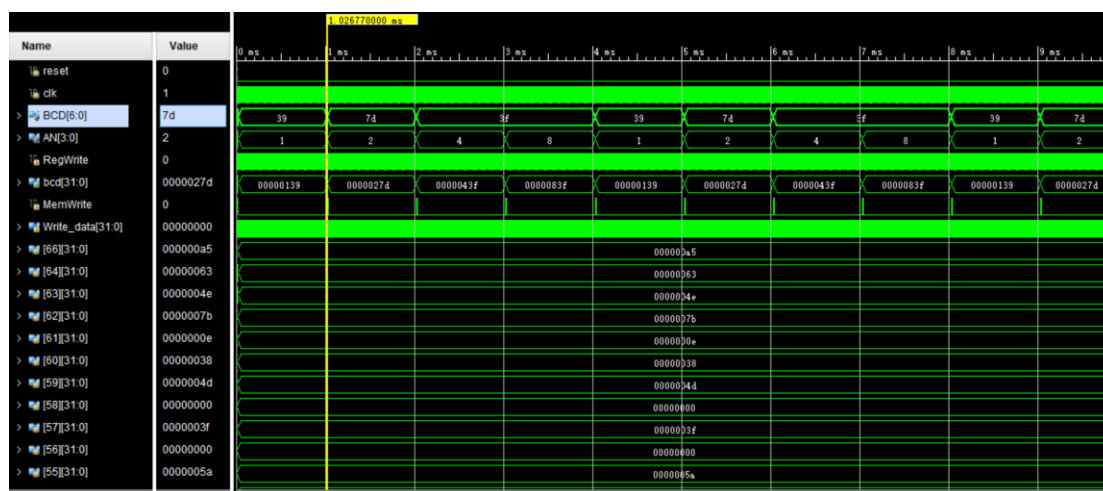
2. 文件清单

文件名	功能
top.v	顶层文件，整合了分频、CPU 和 BCD 显示功能
clk_gen.v	对时钟频率进行二分频
CPU.v	整合流水线各模块为完整的 MIPS 五级流水线
PC.v	计算下一条指令的地址
InstMem.v	指令存储器，存储汇编代码生成的指令
IF_ID.v	IF/ID 级间寄存器，将数据从 IF 阶段传递到 ID 阶段
Control.v	生成控制信号

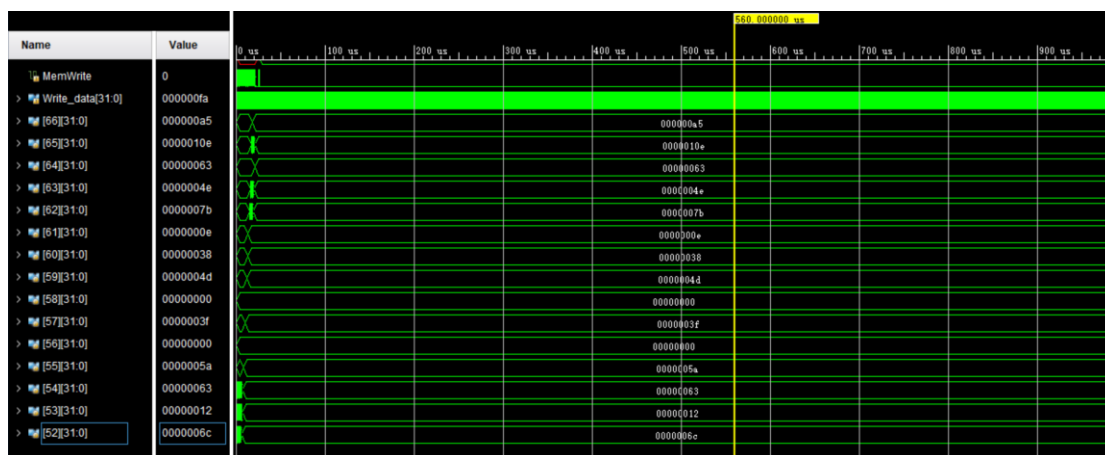
RegisterFile.v	寄存器堆读取和写入
ImmExt.v	立即数扩展
ID_EX.v	ID/EX 级间寄存器，将数据从 ID 阶段传递到 EX 阶段
ALUCtl.v	生成 ALU 控制信号，判断 ALU 操作类型
ALU.v	执行 ALU 运算操作
EX_MEM.v	EX/MEM 级间寄存器，将数据从 EX 阶段传递到 MEM 阶段
DataMem.v	数据存储器，存储原矩阵和 BCD 段码表
MEM_WB.v	MEM/WB 级间寄存器，将数据从 MEM 阶段传递到 WB 阶段
Hazard.v	生成冒险控制信号
Forwarding.v	生成转发信号
top.xdc	约束文件
test_cpu.v	仿真测试文件
sparse_matmul_CPU.asm	汇编代码，实现稀疏矩阵乘法计算和 BCD 显示功能
InstMem_init.txt	由汇编代码生成的二进制机器码指令文件

五、 仿真结果及分析

1. 整体结果



对“计算稀疏矩阵乘法并在 BCD 数码管上显示结果”的程序指令，仿真发现流水线写会的 BCD 编码与汇编代码生成结果一致，且符合每一位轮流快速循环显示 1ms 的特点。因此，认为用汇编代码实现 BCD 依次显示结果矩阵元素部分的功能正常。

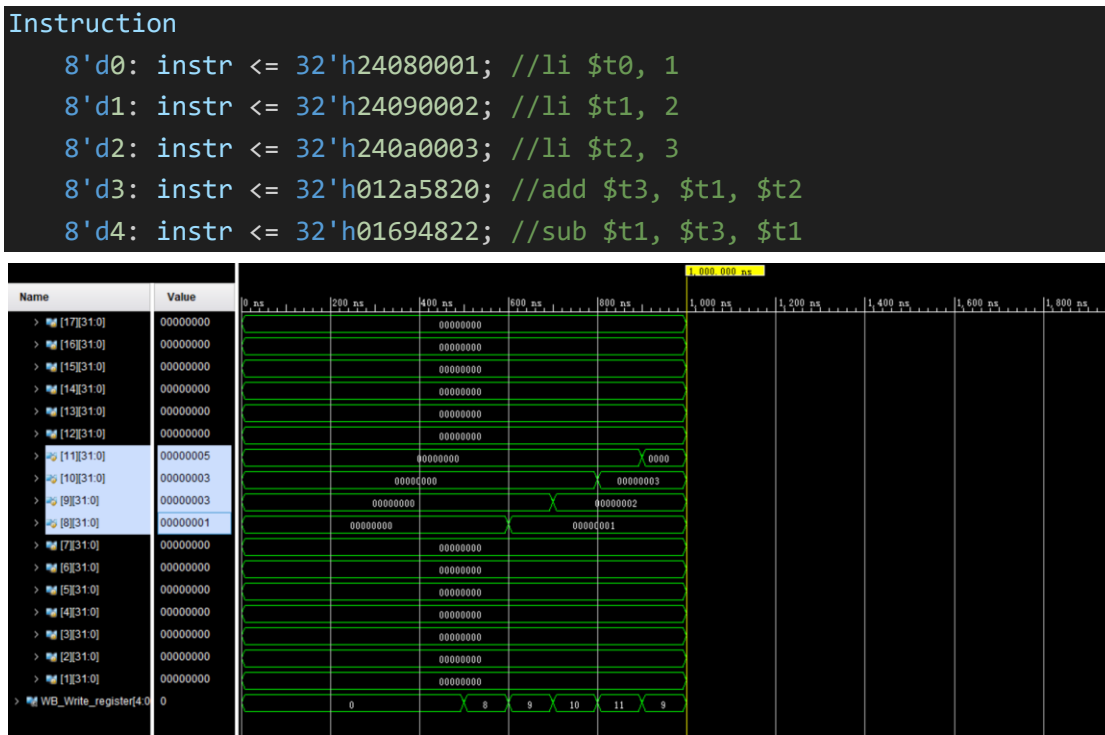


观察局部仿真图像，可以看出流水线很快计算出了稀疏矩阵乘法的正确结果（与参考资料中的汇编大作业结果完全一致）并存储在流水线的数据存储寄存器中。因此，认为稀疏矩阵乘法计算部分的功能正常。

2. 流水线功能验证

修改流水线指令存储器 InstMem，选择放入功能较为单一、且有明确预期结果的指令，可验证搭建的五级流水线基本功能正常。

(1) 验证转发通路没有问题



经仿真确认，DataMem 中存储的结果符合预期，说明流水线的转发通路可正常工作。

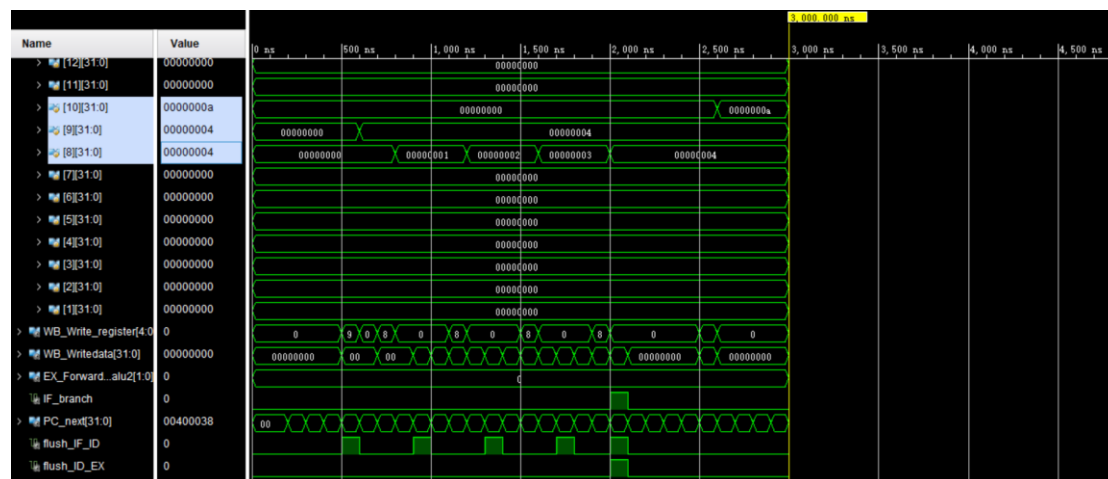
(2) 验证 branch 相关判断、转发没有问题

Instruction

```

8'd0: instr <= 32'h24090004; //li $t1, 4
8'd1: instr <= 32'h11280002; //beq $t1, $t0, end
8'd2: instr <= 32'h21080001; //addi $t0, $t0, 1
8'd3: instr <= 32'h08100001; //j
8'd4: instr <= 32'h240a000a; //li $t2, 10

```



经仿真确认，DataMem 中存储的结果符合预期，说明流水线的分支判断与处理通路可正常工作。

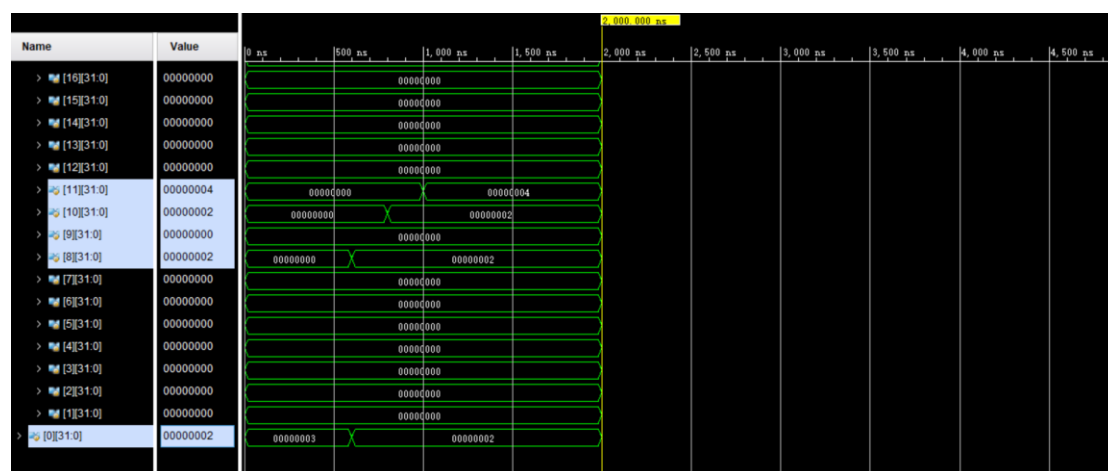
(3) 验证 load-use 相关判断、转发没有问题

Instruction

```

8'd0: instr <= 32'h24080002; //li $t0, 2
8'd1: instr <= 32'had280000; //sw $t0, 0($t1)
8'd2: instr <= 32'h8d2a0000; //lw $t2, 0($t1)
8'd3: instr <= 32'h01485820; //add $t3, $t2, $t0

```

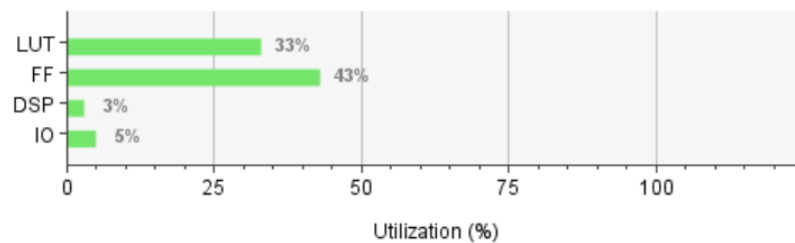


经仿真确认，DataMem 中存储的结果符合预期，说明流水线的 load-use 相关判断与处理功能没有问题。

六、综合情况

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	DSP s (90)	Bonded IOB (250)	BUFGCTRL (32)
top	6871	18028	2317	1120	7946	6871	439	3	13	2
myclk_gen (clk_gen)	1	2	0	0	1	1	1	0	0	0
myCPU (CPU)	6870	18015	2317	1120	7946	6870	438	3	0	0
myALU (ALU)	15	0	0	0	4	15	0	3	0	0
myDataMem (Data...	4352	16395	2176	1056	7544	4352	0	0	0	0
myEX_MEM (EX_ME...	413	260	0	0	301	413	2	0	0	0
myID_EX (ID_EX)	873	168	1	0	303	873	0	0	0	0
myIF_ID (IF_ID)	211	64	0	0	109	211	16	0	0	0
myInstMem (InstMe...	80	0	12	0	32	80	0	0	0	0
myMEM_WB (MEM_...	315	104	0	0	222	315	0	0	0	0
myPC (PC)	35	32	0	0	33	35	0	0	0	0
myRegisterFile (Re...	576	992	128	64	319	576	0	0	0	0

Resource	Utilization	Available	Utilization %
LUT	6871	20800	33.03
FF	18028	41600	43.34
DSP	3	90	3.33
IO	13	250	5.20



Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -4.195 ns	Worst Hold Slack (WHS): 0.119 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): -514.119 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 970	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 35509	Total Number of Endpoints: 35509	Total Number of Endpoints: 18027

Timing constraints are not met.

指定时钟周期为 10ns 时，对移除了分频模块的流水线，可由图上数据计算出：

$$\text{最短时钟周期 } T = 10\text{ns} + 4.195\text{ns} = 14.195\text{ns}$$

$$\text{最高时钟频率 } f = 1/T = 1/14.195\text{ns} = 70.447\text{MHz}$$



由仿真波形图像可知，100ns 时程序开始执行，5560ns 时程序执行完成，时钟周期为 10ns。由此可计算出：

$$\text{执行时间为 } (5560\text{ns} - 100\text{ns}) / 10\text{ns} = 546 \text{ 个周期}$$

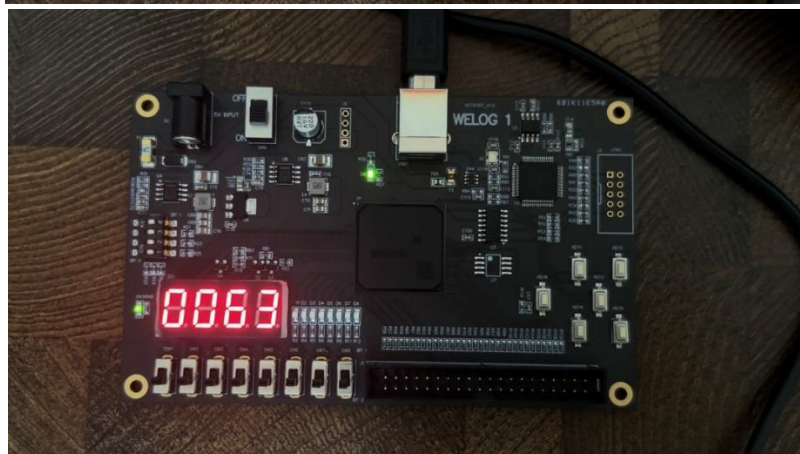
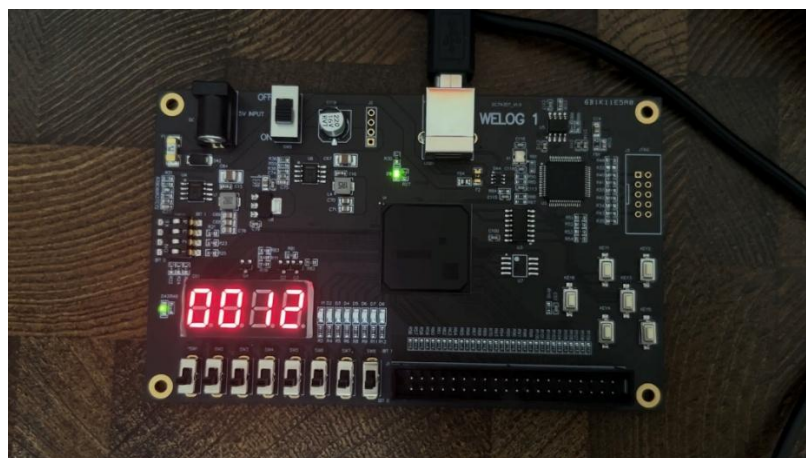
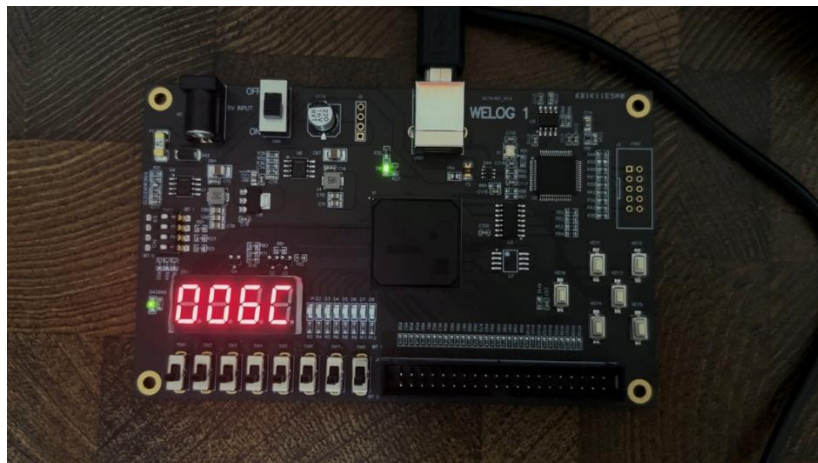
$$\text{CPI} = 546 / 501 = 1.090$$

CPI 接近 1，说明该流水线的资源利用效率较高，这也与汇编代码中 50%以上都是 R 型指令有关。

七、 硬件调试情况

本次实验是在仿真确认无问题后，才烧录比特流文件进行硬件调试的。但我最初发现，硬件并未按照仿真预期进行工作，且没有任何显示。后来发现是因为我搭建的流水线无法支持在 100MHz 的时钟频率下工作。因此，在代码中增加了二分频功能以后，FPGA 硬件板就可以正常工作了，会每隔一秒钟以十六进制形式依次显示结果矩阵的每个元素。

附上前 3 个元素的显示图片：



八、思想体会

本次实验的任务包括搭建流水线、编写汇编代码、生成对应指令后进行仿真、硬件调试几个环节。由于每个环节都可能影响结果的正确性，我通过逐个击破的方式，先依次确认流水线功能正常，汇编代码正常，仿真符合预期后，再进行硬件调试，将每次发生的故障限制在一个环节中，减少了 debug 的工作量。

首先，我在搭建完流水线后，先通过几个简单且只涉及到流水线部分功能的指令，对流水线的功能进行了确认。在流水线设计中，我主要的问题是忘记对部分变量提前进行定义，而直接使用 `assign` 赋值未定义的变量时，会默认该变量为 1bit 数据，导致发生了数据截断，影响结果传递。此外，在进行分支判断处理时，为确定是哪种类型的分支，需要引入 `OpCode`。但由于 `OpCode` 使用需求极少，最初并未设计在级间寄存器中进行传递，但导致使用的 `OpCode` 实际上是下一条指令的，导致判断出错。这些错误都通过观察有预期结果的指令的仿真图像，一步步排查信号出现故障的初始时刻，最终锁定问题所在并及时得到解决。

最开始我在写汇编代码时，也并非是一口气写出了完全正确的显示功能。但通过在存储 BCD 码下一行添加控制台输出，核实正确存储后，就能认定汇编功能正常。

在用汇编代码生成机器码时，我最初因为没有删去 `syscall` 指令和 MIPS 指令集不支持的 `srlv` 和 `sllv` 指令，导致指令进入流水线以后存储了奇怪数据。但由于程序并未对不支持的指令进行报错，导致我在排查问题的环节花费了较长时间。最终通过询问同学得到了启发，解决了这个问题。

在硬件调试中，我主要遇到的问题是仿真正确但板子上不显示任何结果。其原因在于我搭建的流水线无法支持在 100MHz 的时钟频率下工作。因此，在代码中增加了二分频功能以后，FPGA 硬件板就可以正常工作了。

虽然我在搭建流水线和汇编调试过程中遇到了许多问题，但通过发现问题，尝试排查问题并解决问题的过程，也让我对 Verilog 和汇编代码的撰写，以及流水线的工作原理有了更深入的理解。